

40. Šta izaziva pojavu preplitanja?

Stohastično izvršavanje konkurentnih programa može da menja rezultate izvršavanja od slučaja do slučaja, što može dovesti do pojave štetnog preplitanja.

41. Da li preplitanje može uticati na rezultat izvršavanja programa?

Da. Stohastično izvršavanje konkurentnih programa može da menja rezultate izvršavanja od slučaja do slučaja, što može dovesti do pojave štetnog preplitanja.

42. Šta su deljene promenljive?

Promenljive kojima pristupaju više niti ili niti i obrađivači prekida istovremeno (position, list i buffer) se nazivaju deljene promenljive, a klase koje opisuju te promenljive deljene klase.

43. Šta je preduslov očuvanja konzistentnosti deljenih promenljivih?

Deljene klase su napravljene pod pretpostavkom da se rukovanja deljenim promjenljivim obavljaju sekvencijalno odnosno da se operacije deljenih promjenljivih izvrsavaju strogo jedna za drugom. To znaci da novo izvrsavanje bilo koje od operacija deljene promjenljive moze poceti tek nakon zavrsetka prethodno zapocetog izvrsavanja neke od ovih operacija. Na taj nacin svako od ovih izvrsavanja ostavlja i zatice deljene promjenljive u konzistentnom(predvidjenom) stanju.

44. Šta su kritične sekcije?

Tela operacija deljenih klasa ili delovi ovih tela, čije izvršavanje je kritično za konzistentnost deljenih promenljivih, se nazivaju kritične sekcije. Međusobna isključivost kritičnih sekcija se ostvaruje sinhro nizacijom pristupa.

45. Šta je sinhronizacija?

Medjusobna iskljucivost kriticnih sekcija se ostvaruje vremenskim uskladjivanjem njihovih izvrsavanja. Sprovedjenje ovakvog uskladjivanja se naziva sinhronizacija.

46. Koje vrste sinhronizacije postoje?

1. Obicna
2. Uslovna – ostvaruje dodatni uslov u pogledu izvrsavanja kriticnih sekcija

47. Šta je atomski region?

Ako se onemoguće prekidi u kritičnim sekcijama odgovarajuće deljene promenljive, tada u toku izvršavanja ovih kritičnih sekcija nisu moguće ni obrade prekida, pa ni pomenuta štetna preplitanja. Zbog neprekidnosti izvršavanja (nedeljivosti), ovakve kritične sekcije se nazivaju atomski regioni.

48. Šta sužava primenu atomskih regiona?

Onemogućenje prekida u atomskim regionima odlaže obradu novih prekida i usporava reakciju procesora na vanjske događaje, važno je da izvršavanja atomskih regiona budu što kraća. Ovakav zahtev sužava primenljivost onemogućenja prekida kao sredstva za osiguranje međusobne isključivosti kritičnih sekcija.

49. Čemu služi propusnica?

Propusnice su drugi način ostvarenja međusobne isključivosti kritičnih sekcija.

Propusnica može biti slobodna (nedodeljena) ili zauzeta (dodeljena). Podrazumeva se da bez propusnice nije moguć ulazak u kritičnu sekciju deljene promenljive. Zato propusnicu traže sve niti koje se takmiče za ulazak u neku od kritičnih sekcija iste deljene promenljive. Međutim, propusnicu dobija samo jedna od njih i ona ulazi u kritičnu sekciju. Ostale niti zaustavljaju svoju aktivnost i prelaze u stanje "čeka". Svaka od njih ostaje u tom stanju do eventualnog dobijanja tražene propusnice. To se desi tek pošto nit, koja pristupa deljenoj promenljivoj, napusti njenu kritičnu sekciju i vrati propusnicu deljene promenljive. Nit, koja tada dobije propusnicu, odmah prelazi iz stanja "čeka" u stanje "spremna", ali u kritičnu sekciju ulazi tek kada postane aktivna (odnosno, kada se procesor preključi na nju).

50. Šta se dešava sa niti koja zatraži, a ne dobije propusnicu?

Zaustavlja svoju aktivnost i prelazi u stanje "čeka".

51. Šta se dešava kada nit vrati propusnicu?

Nit, koja tada dobije propusnicu, odmah prelazi iz stanja "čeka" u stanje "spremna", ali u kritičnu sekciju ulazi tek kada postane aktivna (odnosno, kada se procesor preključi na nju).

52. Kako se štiti konzistentnost propusnica?

To se može učiniti kroz algoritam ili kroz hardversku podršku. Za to se obično koriste atomski regioni, jer su rukovanja propusnicama kratkotrajna.

53. Šta je isključivi region?

To su kritične sekcije koje međusobnu isključivost ostvaruju korišćenjem propusnica.

54. Šta uvode poželjne osobine konkurentnih programa?

Poželjne osobine konkurentnih programa uvode:

- Tvrdnju isključivanja nepoželjnog (safety property).
- Tvrdnju uključivanja poželjnog (liveness property).

Primer tvrdnje isključivanja nepoželjnog je tvrdnja da se u izvršavanjima konkurentnog programa ne javlja nekonzistentnost date deljene promenljive.

Primer tvrdnje uključivanja poželjnog je tvrdnja da se u toku izvršavanja konkurentnog programa dese svi zatraženi ulasci u dati isključivi region.

55. Po čemu se konkurentno programiranje razlikuje od sekvencijalnog?

Konkurentno programiranje se razlikuje od sekvencijalnog po rukovanju nitima i deljenim promenljivama.

56. Koje prednosti ima konkurentna biblioteka u odnosu na konkurentni programski jezik?

Konkurentna biblioteka omogućuje da se za konkurentno programiranje koristi već postojeći, poznat programski jezik.

57. Kako se opisuju niti?

Opisuju se funkcijom čija se adresa kasnije prosleđuje kao argument poziva konstruktora klase thread.

58. Kako se kreiraju niti?

Rukovanje nitima omogućuje klasa thread. Njen konstruktor je zadužen za kreiranje (stvaranje i pokretanje) niti. Kao argument poziva ovog konstruktora navodi se adresa funkcije koja opisuje nit (njenu aktivnost). . Da ne bi došlo do prevremenog kraja aktivnosti kreirane niti, klasa thread nudi operaciju join(). Ona zaustavlja aktivnost svog pozivaoca sve dok se ne završi aktivnost niti na koju se odnosi ova operacija.

Klasa thread nudi i operaciju detach() pomoću koje se saopštava da je dozvoljen nasilni (prevremeni) kraj aktivnosti niti (da regularan kraj aktivnosti niti, na koju se odnosi ova operacija, može da nastupi i kao posledica kraja aktivnosti njenog procesa).

59. Kada se zauzima propusnica deljene promenljive?

Prije obavljanja svojih izlaznih i ulaznih operacija, na pocetku kritične sekcije.

60. Kada se oslobađa propusnica deljene promenljive?

Nakon obavljanja svojih izlaznih i ulaznih operacija, na kraju kritične sekcije.

61. Kakvu ulogu ima klasa mutex?

Da bi se sprečilo mešanje izlaznih i ulaznih operacije raznih niti, svaka od njih treba da zauzme terminal pre obavljanja svojih izlaznih i ulaznih operacija, a nakon njihovog obavljanja da oslobodi terminal. Pošto pomenuto korišćenje terminala predstavlja kritičnu sekciju, međusobna isključivost ovih kritičnih sekcija se može zasnovati na upotrebi propusnice koja reprezentuje terminal. To omogućuje klasa mutex. Njen objekat predstavlja propusnicu. Sinhronizacija kroz mutex je mehanizam koji omogućava da kontrolišemo koliko niti mogu biti u nekoj ključnoj sekciji koda. Zasnivanje međusobne isključivosti na klasi mutex podrazumeva pozivanje njene

operacije lock() na početku kritične sekcije i pozivanje njene operacije unlock() na kraju kritične sekcije.

62. Kakvu ulogu ima klasa unique_lock?

njen konstruktor poziva operaciju lock(), a destruktor operaciju unlock(). Da bi se to postiglo, kao argument templejt klase unique_lock mora se navesti klasa mutex, a kao argument konstruktora templejt klase unique_lock mora se navesti objekat klase mutex.

```
// unique_lock m;
```

63. Kakvu ulogu ima klasa condition_variable?

Za ostvarenje ovakve uslovne sinhronizacije zadužena je klasa condition_variable. Ova klasa nudi operacije wait() i notify_one().

64. Koje vrste razmene poruka postoje?

Asinhrona i sinhrona.

65. U čemu se razlikuju sinhrona i asinhrona razmena poruka?

Asinhrona - pošiljalac i primalac se ne sreću prilikom razmene poruka (aktivnost pošiljaoca se zaustavlja pri slanju poruka samo kada je komunikacioni kanal pun, dok se aktivnost primaoca zaustavlja pri prijemu poruka samo kada je ovaj kanal prazan). Ako se kapacitet komunikacionog kanala poveća na dve ili više poruka, tada svakom prijemu mogu da prethode dva ili više slanja.

Uz zadržavanje kapaciteta komunikacionog kanala na jednoj poruci, razmena poruka postaje sinhrona, ako se uvek zaustavlja aktivnost niti koja prva započne razmenu poruka, bez obzira da li se radi o pošiljaocu ili primaocu. Aktivnost ove niti ostaje zaustavljena dok i druga nit ne započne razmenu poruka (dok se pošiljalac i primalac na sretnu). Pri tome se podrazumeva da pošiljalac nastavlja svoju aktivnost tek kada primalac preuzme poruku.

66. Opisati problem pet filozofa.

Zauzimanje više primeraka resursa iste vrste, neophodnih za aktivnost svake niti iz neke grupe niti, predstavlja tipičan problem konkurentnog programiranja. On se, u literaturi, ilustruje primerom problema pet filozofa (dining philosophers). Svaki od njih provodi život razmišljajući u svojoj sobi i jedući u zajedničkoj trpezariji. U njoj se nalazi pet stolica oko okruglog stola sa pet tanjira i pet viljuški između njih (Slika 2.2). Pošto se, po želji filozofa, u trpezariji služe uvek špagete, svakom filozofu su za jelo potrebne dve viljuške (one predstavljaju resurse koje filozofi zauzimaju). Ako svi filozofi istovremeno ogladne, uđu u trpezariju, sednu na svoje mesto za stolom i uzmu viljušku levo od sebe, tada nastupa mrtva petlja (deadlock), s kobnim ishodom po život filozofa.

67. Kako bi izgledala verzija problema pet filozofa koja bi se realistično mogla 2 sresti tokom razvoja softvera?

Npr. 5 računara je povezano na centralnu bazu podataka, gdje su umreženi tako da susjedna 2 računara ne mogu u isto vrijeme da koriste lan konekciju.

68. Šta je mrtva petlja?

Mrtva petlja nastaje kada više niti/procesa pristupaju deljenim resursima koji su međusobno zavisni i kojima se pristupa u režimu isključivosti. Ono što ovi uslovi omogućavaju jeste da se procesi/niti dovedu u situaciju gde za svaku nit/proces važi da čeka neku drugu nit/proces i grafik ovih zavisnosti formira zatvoren ciklus. To znači da niti/procesi nisu aktivne no, su u istanju čekanja i iz tog stanja nikako ne mogu izaći: odatle naziv 'mrtva petlja.'

69. Po čemu se živa petlja razlikuje od mrtve petlje?

U slučaju neblokirajuće sistemske operacije zaključavanja, procesi upadaju u beskonačnu petlju (starvation), pokušavajući da zaključaju datoteku, koju je zaključao drugi proces. Ovakav oblik međusobne zavisnosti procesa se naziva i živa petlja (livelock). Ona se, po svom ishodu, suštinski ne razlikuje od mrtve petlje.

70. Koji uslovi su potrebni za pojavu mrtve petlje?

1. zauzimani resursi se koriste u režimu međusobne isključivosti
2. resursi se zauzimaju jedan za drugim, tako da proces, nakon zauzimanja izvesnog broja resursa, mora da čeka da zauzme preostale resurse
3. resurse oslobađaju samo procesi koji su ih zauzeli
4. postoji cirkularna međuzavisnost procesa (prvi proces čeka oslobađanje resursa koga drži drugi proces, a on čeka oslobađanje resursa koga drži treći proces, i tako redom do poslednjeg procesa iz lanca procesa, koji čeka oslobađanje resursa koga drži prvi proces)

71. Kako se u praksi tretira problem mrtve petlje?

- Postoje četiri pristupa tretiranja problema mrtve petlje:
 - sprečavanje (prevention) pojave mrtve petlje (onemogućavanje važenja nekog od četiri neophodna uslova za njenu pojavu)
 - izbegavanje (avoidance) pojave mrtve petlje
 - otkrivanje (detection) pojave mrtve petlje i oporavak (recovery) od nje
 - ignorisanje pojave mrtve petlje.

72. Na čemu se temelji sprečavanje mrtve petlje?

Temelji se na onemogućavanju važenja nekog od četiri neophodna uslova za njenu pojavu.

Kod sprečavanja pojave mrtve petlje, važenje prvog uslova obično nije moguće sprečiti, jer se resursi najčešće koriste u režimu međusobne isključivosti.

Važenje drugog uslova se može sprečiti, ako se unapred zna ko liko treba resursa i ako se oni svi zauzmu pre korišćenja.

Pri tome, neuspeh u zauzimanju bilo kog resursa dovodi do oslobađanja prethodno zauzetih resursa, što je moguće učiniti bez posledica, jer nije započelo njihovo korišćenje.

Važenje trećeg uslova se obično ne može sprečiti, jer najčešće ne postoji način da se zauzeti resurs privremeno oduzme pro cesu.

I konačno, važenje četvrtog uslova se može sprečiti, ako se resursi uvek zauzimaju u unapred određenom redosledu, koji isključuje mogućnost cirkularne međuzavisnosti procesa (primer pozitivnog rešenja 5 filozofa)

73. Šta karakteriše izbegavanje mrtve petlje?

Izbegavanje pojave mrtve petlje zahteva poznavanje podataka:

- o maksimalno mogućim zahtevima za resursima
- ukupno postavljenim zahtevima za resursima
- stanju resursa

Podrazumeva se da se udovoljava samo onim zahtevima za koje se proverom ustanovi da, nakon njihovog ispunjavanja, postoji redosled zauzimanja i oslobađanja resursa u kome se mogu zadovoljiti maksimalno mogući zahtevi svih procesa. Praktična vrednost ovoga pristupa nije velika, jer se obično un apred ne znaju maksimalno mogući zathevi procesa za resur sima, a to je neophodno za proveru da li se može udovoljiti po jedinim zahtevima. Sem toga, ovakva provera je komplikovana, a to znači i neefikasna.

74. Šta karakteriše otkrivanje i oporavak od mrtve petlje?

Otkrivanje pojave mrtve petlje se zasniva na sličnom pristupu kao i izbegavanje pojave mrtve petlje. U ovom slučaju se proverava da li postoji proces, čijim zahtevima se ne može udovoljiti ni za jedan redosled zauzimanja i oslobađanja resursa. Pored komplikovanosti ovakve provere, problem je i šta učiniti, kada se i otkrije pojava mrtve petlje. Ako se resursi ne mogu privremeno oduzeti od procesa, preostaje jedino uništavanje procesa, radi oslobađanja resursa koje oni drže. Međutim, to nije uvek prihvatljiv zahvat. Zbog toga ni ovaj pristup nema veliki praktični značaj.

75. Šta karakteriše ignorisanje mrtve petlje?

Ignorisanje pojave mrtve petlje je pristup koji je najčešće primenjen u praksi, jer se ovaj problem ne javlja tako često da bi se isplatilo da ga rešava operativni sistem. Prema tome, kada se mrtva petlja javi, na korisniku je da se suoči sa ovim problemom i da ga reši na način, koji je primeren datim okolnostima.

76. Opisati problem čitanja i pisanja.

ao što je rukovanje bankovnim računima. Bankovni računi pripadaju komitentima banke i sadrže ukupan iznos novčanih sredstava svakog od komitenata. U najjednostavnijem slučaju, rukovanje bankovnim računima se svodi: na prenos sredstava(s jednog računa na drugi) i na provjeru (stanja svih) računa. Prenos sredstava obuhvata četiri koraka:

1. Čitanje stanja računa s koga se prenose sredstva
 2. Pisanje novog stanja na ovaj račun. Novo stanje se dobije umanjivanjem pročitano stanja za prenošeni iznos
 3. Čitanje stanja računa na koji se prenose sredstva.
 4. Pisanje novog stanja na račun na koji se prenose sredstva. Novo stanje se dobije uvećavanjem pročitano stanja za prenošeni iznos
- Pretpostavka je da se prenosi mogu vršiti samo između pomenutih računa, tj. ukupna suma njihovih stanja je nepromjenljiva. Provjera računa se svodi na čitanja, jedno za drugim, stanja svih računa, radi njihovog sumiranja. Ispravnost prenosa sredstava zavisi od očuvanja konzistentnosti stanja svih računa, za šta je neophodna međusobna isključivost raznih prenosa sredstava. U suprotnom, moguće su razne greške. Dakle, važno je:

- da je za ispravnost prenosa bitno da prenosi budu međusobno isključivi.
 - da je za ispravnost provjera bitno da provjere i prenosi budu međusobno isključivi.
- Pošto prenosi sadrže pisanja, a provjere samo čitanja, sledi da operacije sa pisanjem moraju biti međusobno isključive, kao što moraju biti međusobno isključive operacije sa pisanjem i operacije sa čitanjem. Za operacije koje sadrže samo čitanja međusobna isključivost nije potrebna.

77. Šta karakteriše semafor?

Semafor je sredstvo za sinhronizaciju niti (reguliše prolaz niti kroz kritičnu sekciju) i zasniva se na ideji saobraćajnog semafora koji reguliše ulazak vozova u stanicu sa jednim kolosijekom.

78. Koje operacije su vezane za semafor?

stop() i resume(). Operacija stop() se poziva na početku kritične sekcije, radi provere da li je moguć ulazak u kritičnu sekciju. U okviru ove provere, stanje semafora se menja da bi se onemogućio novi ulazak u kritičnu sekciju. Ako ulazak u kritičnu sekciju nije

moć (ako stanje semafora nije veće od nula), aktivnost niti poziva ova operacija se zaustavlja. Operacija resume() se poziva na kraju kritične sekcije, radi izmene stanja semafora i omogućavanja da u kritičnu sekciju uđe jedna od niti koje čekaju pred kritičnom sekcijom (ako takve niti postoje).

79. Kako semafor obezbeđuje sinhronizaciju međusobne isključivosti?

Sinhronizacija niti, koju omogućuje semafor, se zasniva na zaustavljanju aktivnosti niti, kao i na omogućavanju nastavljanja njihove aktivnosti. Ulazak niti u kritičnu sekciju zavisi od stanja semafora. Kada stanje semafora dozvoli ulazak niti u kritičnu sekciju, pri ulasku se semafor prevodi u stanje koje onemogućuje ulazak druge niti u kritičnu sekciju. Ako se takva nit pojavi, njena aktivnost se zaustavlja pred kritičnom sekcijom. Pri izlasku niti iz kritične sekcije semafor se prevodi u stanje koje dozvoljava novi ulazak u kritičnu sekciju i ujedno omogućuje nastavak aktivnosti niti koja najduže čeka na ulaz u kritičnu sekciju (ako takva nit postoji).

80. Kako se obično implementira semafor?

Semafori se obično implementiraju u okviru operativnog sistema i tada se njihova implementacija obično zasniva na (kratkotrajnom) onemogućenju prekida.

81. U čemu se semafori razlikuju od isključivih regiona?

Semafori i isključivi regioni predstavljaju dva različita pristupa sinhronizaciji. Isključivi regioni su prilagođeni objektno orijentisanom programiranju, dok su semafori prilagođeni procedurnom programiranju.

82. Koji semafori postoje?

- Binarni semafori
- Generalni semafori
- Raspodijeljeni binarni semafor (posebna vrsta binarnog)

83. Šta karakteriše binarni semafor?

Semafor čije stanje ne može preći vrednost 1 se zove binarni semafor (binary semaphore). On omogućuje ostvarenje sinhronizacije međusobne isključivosti, ako se njegovo stanje inicijalizuje na vrednost 1.

84. Šta karakteriše raspodeljeni binarni semafor?

On se realizuje pomoću više binarnih semafora, za koje važi ograničenje da suma njihovih stanja ne može preći vrednost 1.

85. Šta karakteriše generalni semafor?

Semafor, čije stanje može sadržati vrednost veću od 1, se naziva generalni semafor.

86. Šta omogućuje raspodeljeni binarni semafor?

Pomoću raspodeljenog binarnog semafora se ostvaruje uslovna sinhronizacija tako što se na ulazu u svaku kritičnu sekciju poziva operacija `stop()` jednog od njegovih binarnih semafora, a na izlazu iz nje operacija `resume()` tog ili nekog od preostalih binarnih semafora. Na taj način najviše jedna nit se može nalaziti najviše u jednoj od pomenutih kritičnih sekcija, jer su stanja svih semafora manja od vrednosti 1 za vreme njenog boravka u dotičnoj kritičnoj sekciji.

87. Šta omogućuje binarni semafor?

On omogućuje ostvarenje sinhronizacije međusobne isključivosti, ako se njegovo stanje inicijalizuje na vrednost 1. Tada su njegove operacije `stop()` i `resume()` slične operacijama `lock()` i `unlock()` klase `mutex`. Ako se stanje binarnog semafora inicijalizuje na vrednost 0, tada su njegove operacije `stop()` i `resume()` slične operacijama `wait()` i `notify_one()` klase `condition_variable`.

88. Šta omogućuje generalni semafor?

On omogućuje ostvarenje uslovne sinhronizacije prilikom rukovanja resursima. Pozitivno stanje generalnog semafora može predstavljati broj slobodnih primeraka nekog resursa. Zahvaljujući tome, zauzimanje primerka resursa se može opisati pomoću operacije `stop()`, a njegovo oslobađanje pomoću operacije `resume()` generalnog semafora.

89. Koje su prednosti i mane semafora?

Mehanizam semafora je jednostavan i efikasan, ali je njegova mana što raspodeljeni binarni semafori nisu baš najpodesnije sredstvo za opisivanje uslovne sinhronizacije.